

Prog 4 – pótZH

A feladatsort figyelmesen olvasd el!

A ZH-t zip fájlként kell feltölteni az alábbi formátumban: `név_neptun_kód_prog4_potzh.zip`

Az alkalmazáserver helye a labor gépein: <C:\apache-tomcat>

A ZH feladat leírása és kiinduló projektje mellett található egy **auditalo.exe** állomány, amit első lépésként tölts le.

Hozz létre egy mappát az alábbi néven: `<név_neptun_kód>_prog4_potzh`

Ebbe a létrehozott mappába másold be az **auditalo.exe** állományt, majd indítsd el, ez létre fog hozni két mappát: **zh**, **zh_audit**

Ha a Windows blokkolná a futtatást, akkor kattints a felugró ablakban a **További információ** linkre, majd a **Futtatás mindenképpen** opcióra, ezután megnyílik egy command window, **ezt ne zárd be, a teljes ZH alatt futnia kell.**

Ezt követően tölts le a kiinduló projektet, majd csomagold ki (vagy másold át a pot-feladat mappát) a **zh** mappába és itt dolgozz (innen nyisd meg Intellij Idea-ban).

A **zh_audit** mappa tartalmához TILOS nyúlni és annak módosítása is TILOS.

Nem forduló kód (maven build használatával) vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) és chat alkalmazások megnyitása vagy használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott megoldás alapján) szintén 0 pontos ZH-t eredményez.

Az auditáló futtatásának hiánya, annak idő előtti leállítása, a fájljainak módosítása 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér (ez pom.xml-re is vonatkozik, csak az legyen behúzva, amire az adott alkalmazásnak szüksége van).

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git repo hiánya esetén a ZH 0 pontos, commitálás nélkül beadott kód 0 pontot ér.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során.

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így az adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

A **zh** mappán kívüli található megoldások nem lesznek figyelembe véve, arra 0 pont jár.

Beadáskor egyetlen tömörített (ZIP) állományt kell feltölteni. Ehhez a korábban létrehozott `<név_neptun_kód>_prog4_potzh` mappát tömörítsd. A feltöltött állománynak tartalmaznia kell a **zh** és **zh_udit** mappákat, valamint az auditáló programot.

Ellenőrizd, hogy a mariadb-java-client-3.1.3.jar megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsd le az internetről és helyezd el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítod a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolathoz szükséges resource definíciója, **amit szükséges kiegészíteni a hiányzó értékekkel** (ha már van másik resource felvéve azt töröld ki):

```
<Resource name="jdbc/PotZH_pet_MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/potzh_pet_database" username="" password=""
maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban két felhasználó van.

user1: neki searcher jogosultsága van

user2: neki searcher és creator jogosultsága van

Mind a kettőnek "Password111" a jelszava. A jelszavak Bcrypttel vannak elforgatva.

Rövid ismertető az alkalmazásokról:

Az alkalmazások állatok listázását és mentését végzik.

Két külön alkalmazásunk van, az egyik a potzh-feladat alkalmazás, ami kommunikál az adatbázissal.

Ennek az alkalmazásnak a felületei jsp-vel készültek, és új állatok rögzítésére és azonosító alapján állatok lekérdezésére szolgáló felülete van.

Ez az alkalmazás security-t használ, az index felületet kivéve a többi csak a megfelelő jogosultsággal rendelkező felhasználó tudja elérni.

Szintén ennek az alkalmazásnak vannak api endpointjai is, amik segítségével REST-en keresztül tudunk adatokat lekérdezni.

Az alkalmazásban található egy webservice, amin keresztül az adatbázis tábla tartalmát lehet lekérdezni.

A második alkalmazásunk a potzh-listazo-feladat alkalmazás, aminek egy listázó felülete van, ami nem jsp-vel készült, hanem a felületet servlet generálja.

Ebben az alkalmazásban nincs security.

A két alkalmazás webservicen keresztül kommunikál egymással, a potzh-feladat kiájánl egy listázó webservice endpointot a potzh-listazo-feladat alkalmazás számára, ami ezen keresztül le tudja kérdezni az adatokat.

Feladat leírás:

Git (potzh-feladat és potzh-listazo-feladat)

- Hozz létre egy git repository-t, másold be a kapott alkalmazást, majd a későbbiek során ebbe a repository-ba hozd létre a másik alkalmazást is.
- Készíts egy develop branchet, majd ezen a branchen dolgozz.
- A kapott projekt fájlokat commitáld el egyben.
- Minden feladat megoldását külön-külön commitáld el, figyelj a beszédes commit message-kre.

A kiadott projekt egy minimális pom.xml-t és az entityket tartalmazza, ezt egészítsd ki, hogy a tomcat szerveren web alkalmazásként futtatható legyen.

Az adatbázisban található táblákat és az entityket használva készítsd el az alkalmazás repository rétegét, ahol lehet menteni, frissíteni, listázni és adott azonosítóval történő lekérdezést végezni. A szerepkörök és a felhasználók esetén szintén készítsd el a lekérdezéseket a repository rétegben.

Készítsd el a repository metódushoz tartozó service rétegbeli metódusokat.

Készítsd el a webservice-t, ami az állatok listáját fogja átadni a listázó alkalmazás számára.

Készítsd el az api endpointokat a service rétegben található metódusokhoz.

A potzh-feladat alkalmazásban készíts egy index oldalt.

Készíts egy rögzítő felületet, ahol új állatot lehet rögzíteni.

Készíts egy lekérdező oldalt, ahol található egy input mező, egy Lekérdezés gomb, és egy Legutóbb lekérdezett gomb.

A Lekérdezés gombra kattintva az input mezőbe beírt azonosító alapján kérdezd le az adott állatnak az adatait.

Ha nincs megadva azonosító, akkor írja ki a felületre a kereső mező alá, hogy hiányzik az azonosító.

Ha van megadva azonosító, de nem található ilyen állat, akkor írja ki a kereső mező alá, hogy az állat nem található.

Ha talált állatot az azonosítóval, akkor írja ki az adatait a felületre táblázatos formában.

Kereséskor, ha található állat az azonosítóval, akkor az kerüljön elmentésre egy cookie-ba.

A Legutóbb lekérdezett gombra kattintva kérdezd le a cookie-ban található állat azonosító alapján az állatot, a megjelenítéshez készíts egy egyedi tag libraryt és ezt használva jelenítsd meg az adatait a felületen a kereső mező fölött (tag-ből ne hívj servicet!).

Ha nem található a cookie-ban érték akkor írja ki, hogy még nem történt keresés.

Az oldalak tetején legyen egy tag, amiben a navigáció van megvalósítva az oldalak között, illetve legyen itt egy kijelentkezés link is, ami a kijelentkeztetést fogja végezni.

Készítsd el az alkalmazáshoz az autentikációt.

Működése:

Legyen FORM alapú bejelentkezés.

Az index oldal legyen elérhető bejelentkezés nélkül.

A REST apik és a WS legyenek elérhetőek bejelentkezés nélkül.

A kereső oldal csak „searcher” szerepkörrel legyen elérhető.

A rögzítő oldal csak „creator” joggal legyen elérhető.

Legyen hibás bejelentkezésnek saját oldala.

Korábban az oldalak tetejére elhelyezett tag-ben elkészített kijelentkezés linkre kattintva jelentkeztesen ki.

Készítsd el a potzh-listazo-feladat alkalmazást ezen a néven, ami szintén egy tomcaten futtatható webes alkalmazás.

A korábban a potzh-feladat alkalmazásban elkészített webservice-t használva kérdezd le a listát és jelenítsd meg azt a felületen, a felületet servlet generálja.